Homework 1

羊瑞
2300013091

2026 年 3 月 15 日

Problem 1

1.2 考虑下述选择排序算法：

算法 ModSelectSort

输入：n 个不等的整数的数组 $A[1..n]$

输出：按递增次序排序的 A

```
for i←1 to n−1 do
    for j←i+1 to n do
        if A[j]<A[i] then A[i]↔A[j]
```

问：

(1) 最坏情况下该算法做多少次比较运算？

(2) 最坏情况下该算法做多少次交换运算？这种情况在什么输入条件下发生？

Solution.

(1) 最坏情况下比较次数为：$W(n) = 1 + 2 + ... + (n-1) = \frac{n(n-1)}{2}$.

(2) 最坏情况下, 交换次数与比较次数相等, 为 $\frac{n(n-1)}{2}$ 次, 此时输入的 n 个数从前到后严格单调递减. □

Problem 2

1.6 阅读关于下述算法 A 的伪码，说明该算法求解的是什么问题，并计算该算法所做的乘法运算（*）和加法运算次数.

算法 A

输入：数组 $P[0..n]$，实数 x

输出：y

```
y←P[0]; power←1
for i←1 to n do
    power←power*x
    y←y+P[i]*power
return y
```

Solution.

该算法求的是多项式 $P(x) = P[0] + P[1] * x + P[2] * x^2 + ... + P[n] * x^n$ 的值, 乘法运算次数 2n, 加法次数 n. □

Problem 3

1.8 已知 L 是含有 n 个元素并且排好序的数组，x 在 L 中. 如果 x 出现在 L 中第 i 个 $(i=2,3,\cdots,n)$ 位置的概率是在前一个位置概率的一半，当 n 充分大时，估计下述查找算法平均情况下的时间复杂度 $A(n)$. 只需给出近似值.

算法：顺序查找

```
j←1
while j≤n and x>L[j] do
    j←j+1
if x<L[j] or j>n
then j←0
```

Solution.

设 x 出现在第一个位置的概率为 p, 则 $1 = p + \frac{p}{2} + \frac{p}{2^2} + ... + \frac{p}{2^n} + ...$, 故 $p = \frac{1}{2}$.

设算法结束所需次数为随机变量 K, 有 $P(K <= k) = p + \frac{p}{2} + ... + \frac{p}{2^{k-1}} = \frac{1}{2} + \frac{1}{4} + ... + \frac{1}{2^k}$, 进而 $P(K = k) = \frac{1}{2^k}$.

$A(n) = \frac{1}{2} * 1 + \frac{1}{4} * 2 + \frac{1}{8} * 3 + ... + \frac{1}{2^n} * n \approx \frac{1}{2} * 4 = 2.$ □

Problem 4

1.12 证明定理 1.4：对每个 $b>1$ 和每个 $a>0$，有 $\log_b n = o(n^a)$.

Solution.

证明：$lim_{n\to\infty} \frac{\log_b n}{n^a} = lim_{n\to\infty} \frac{\frac{1}{n \log b}}{an^{a-1}} = lim_{n\to\infty} \frac{1}{a \log b} \frac{1}{n^a} = 0$, 所以 $\log_b n = o(n^a)$. □

Problem 5

1.15 考虑下面每对函数 $f(n)$ 和 $g(n)$，如果它们的阶相等，则使用 Θ 记号；否则使用 O 记号表示它们的关系.

(1) $f(n)=(n^2-n)/2$, $g(n)=6n$

(2) $f(n)=n+2\sqrt{n}$, $g(n)=n^2$

(3) $f(n)=n+n\log n$, $g(n)=n\sqrt{n}$

(4) $f(n)=2\log^2 n$, $g(n)=\log n+1$

(5) $f(n)=\log(n!)$, $g(n)=n^{1.05}$

Solution.

$(2) f(n) = O(g(n))$

$(4) g(n) = O(f(n))$ □

Problem 6

1.18 对以下函数，按照它们的阶从高到低排列；如果 $f(n)$ 与 $g(n)$ 的阶相等，表示为 $f(n)=\Theta(g(n))$.

$$2^{\sqrt{2\log n}},\quad n\log n,\quad \sum_{k=1}^{n}\frac{1}{k},\quad n2^n,\quad (\log n)^{\log n},\quad 2^{2n},\quad 2^{\log\sqrt{n}}$$

$$n^3,\quad \log(n!),\quad \log n,\quad \log\log n,\quad n^{\log\log n},\quad n!,\quad n,\quad \log 10^n$$

Solution.

将函数按阶从高到低排列：

$n!, 2^{2n}, n2^n, n^{\log\log n} = \Theta((\log n)^{\log n}), n^3, n\log n = \Theta\log(n!), n = \Theta(\log 10^n), 2^{log\sqrt{n}}, 2^{\sqrt{2\log n}}, \log n = \Theta(\sum_{k=1}^{n}\frac{1}{k}), \log\log n$ □

Problem 7

1.19 求解以下递推方程：

(1) $\begin{cases} T(n)=T(n-1)+n^2 \\ T(1)=1 \end{cases}$

(2) $\begin{cases} T(n)=9T(n/3)+n \\ T(1)=1 \end{cases}$

(3) $\begin{cases} T(n)=T\left(\frac{n}{2}\right)+T\left(\frac{n}{4}\right)+cn, c\text{ 为常数} \\ T(1)=1 \end{cases}$

(4) $\begin{cases} T(n)=T(n-1)+\log 3^n \\ T(1)=1 \end{cases}$

(5) $\begin{cases} T(n)=5T(n/2)+(n\log n)^2 \\ T(1)=1 \end{cases}$

(6) $\begin{cases} T(n)=2T\left(\frac{n}{2}\right)+n^2\log n \\ T(1)=1 \end{cases}$

(7) $\begin{cases} T(n)=T(n-1)+\frac{1}{n} \\ T(1)=1 \end{cases}$

(8) $T(n)=T(n-1)+\log n$，估计 $T(n)$ 的阶

Solution.

(2) 带入主定理: $a=9, b=3, f(n)=n.n^{\log_b a}=n^2$. f(n) 阶更低, 所以 $T(n)=\Theta(n^2)$

(4)$T(n)=T(n-1)+\log 3^n=T(n-2)+\log 3^n+\log 3^{n-1}=...=n\log 3+(n-1)\log 3+...+2log3+1=\Theta(n^2)$

(6) 带入主定理: $a=2, b=2, f(n)=n^2\log n.n^{\log_b a}=n$. f(n) 的阶更高, $af(\frac{n}{b})=2f(\frac{n}{2})=2(\frac{n}{2})^2\log\frac{n}{2}=\frac{n^2}{2}(\log n-1)<=\frac{n^2}{2}\log n<=\frac{1}{2}f(n)$. 取 $c=\frac{1}{2}$ 满足条件. 所以 $T(n)=\Theta(n^2\log n)$. □

Problem 8

1.21 设原问题的规模是 n，从下述 3 个算法中选择一个最坏情况下时间复杂度最低的算法，简要说明你的理由.

算法 A：将原问题划分规模减半的 5 个子问题，递归求解每个子问题，然后在线性时间将子问题的解合并得到原问题的解.

算法 B：先递归求解 2 个规模为 $n-1$ 的子问题，然后在常量时间内将子问题的解合并.

算法 C：将原问题划分规模为 $n/3$ 的 9 个子问题，递归求解每个子问题，然后在 $O(n^3)$ 时间将子问题的解合并得到原问题的解.

Solution.

A: $T(n)=5T(\frac{n}{2})+O(n), T(n)=\Theta(n^{log_2 5})$

B: $T(n)=2T(n-1)+O(1), T(n)=\Theta(2^n)$

C: $T(n) = 9T(\frac{n}{3}) + O(n^3), T(n) = \Theta(n^3)$

综上, 选择 A. $\square$